ATLAS

Desarrollo de un agente de inteligencia artificial conversacional con capacidad de actuación

Sami González Kamel
Grupo: 2nBATA
Centro: IESTARRAGONA
Tutor: Rafael Roca Campos

21 de octubre de 2026
Memoria final

Resumen

En este trabajo investigué cómo construir un agente conversacional capaz de actuar sobre su entorno e integrarlo en un dispositivo físico accesible. El objetivo era ir más allá de un chat: quería que ATLAS interpretara peticiones, utilizara herramientas y conservara información relevante con suficiente rapidez para mantener una conversación.

Seguí un proceso experimental basado en prototipos, documentación técnica, pruebas y revisión de errores. Inicialmente utilicé OpenClaw para conectar el modelo, la memoria y las herramientas. Esta estructura validó la idea, pero la cadena de transcripción, respuesta y síntesis de voz añadía demasiada espera. Por eso desarrollé una arquitectura propia centrada en GPT Realtime y herramientas directas.

El prototipo integra una Raspberry Pi 5, una pantalla táctil, un micrófono y altavoces. Dispone de WebScreen, chat de terminal, aplicación Android, integración con WhatsApp y un portal web protegido. Incorpora rutinas y un catálogo de 323 expresiones. La carcasa quedó completada como un modelo tridimensional digital.

Las pruebas comprueban la viabilidad del conjunto y muestran la diferencia entre recibir el primer fragmento de respuesta y obtener un resultado útil. Registré recorridos de voz, texto y control de Android, comparé versiones con 400 peticiones y preparé una segunda batería de 400 interacciones nuevas. Las acciones potencialmente molestas se aislaron en un entorno virtual.

La conclusión principal es que la experiencia depende tanto de la coordinación del sistema como del modelo escogido. ATLAS es un prototipo funcional que combina conversación, memoria, herramientas y hardware propio, aunque continúa dependiendo de servicios externos para la conversación.

Abstract

In this project, I investigated how to build a conversational agent that can act on its environment and run through an accessible physical device. My aim was to go beyond a chat interface: ATLAS should interpret requests, use tools and retain relevant information quickly enough to support a conversation.

I followed an experimental process based on prototypes, technical documentation, testing and error analysis. I initially used OpenClaw to connect the model, memory and tools. This validated the idea, but transcription, response and speech synthesis introduced too much delay. I therefore developed an independent architecture centred on GPT Realtime and direct tool execution.

The prototype combines a Raspberry Pi 5, touchscreen, microphone and speakers. It provides WebScreen, a terminal chat, an Android application, WhatsApp integration and a protected web portal. It also includes routines for repetitive tasks and a catalogue of 323 expressions. The enclosure was completed as a digital three-dimensional model.

The tests support the feasibility of the system and show the difference between receiving the first response fragment and obtaining a useful result. I recorded voice, text and Android control flows, compared versions with 400 requests and prepared a second battery of 400 new interactions. Potentially disruptive actions were isolated in a virtual environment.

The main conclusion is that coordination between components matters as much as model capability. ATLAS is a working prototype that combines conversation, memory, tools and dedicated hardware, although it still depends on external services for its main conversational functions.

Agradecimientos

Quiero agradecer el apoyo de las personas que me han acompañado durante este trabajo.

A mi padre, por su apoyo moral, por la supervisión de la memoria escrita y por su ayuda general durante el proyecto.

Al instituto, por brindarme la posibilidad de utilizar la impresora 3D para el proyecto.

A Pedro Jesús Ortega Papaseit, por su ayuda con la impresora 3D y por la corrección de la memoria escrita.

Y a Rafael Roca Campos, por los consejos y las ideas que me ha aportado a lo largo del TDR.

A Alex Viejo, coordinador del Grado de Ingeniería Informática de la URV, por revisar el prototipo y la propuesta inicial del proyecto, y por sus consejos.

Índice

Resum 2

Abstract 3

Agradecimientos 4

Introducción 7

Objetivos 8

Antecedentes 9

I. INFORMÁTICA — El agente y su software 10

1. Concepto del agente conversacional 10

1.1. Definición de agente 10

1.2. El nombre y la identidad de ATLAS 10

1.3. Desarrollo propio y componentes reutilizados 11

2. Evolución de OpenClaw a Realtime 12

2.1. Base del prototipo inicial 12

2.2. El flujo de voz inicial 13

2.3. Transcripción en tiempo real y preámbulos 13

2.4. Motivos del cambio de arquitectura 14

2.5. Componentes conservados y retirados de OpenClaw 15

3. Modelos, contexto y herramientas 15

3.1. Concepto de token 15

3.2. Contexto y límites de memoria 16

3.3. Memoria entre sesiones 17

3.4. Herramientas y resultados verificables 17

4. Arquitectura propia de ATLAS 18

4.1. Componentes y distribución de funciones 18

4.2. Native Broker y preparación de las sesiones 19

4.3. Organización de archivos y contexto 19

4.4. Rutinas para tareas repetitivas 20

4.5. Perfiles y límites de capacidades 20

5. Interfaces y conexión con otros dispositivos 21

5.1. WebScreen y conversación por voz 21

5.2. Voz nativa y motores alternativos 22

5.3. El chat de terminal como herramienta de pruebas 23

5.4. Aplicación Android y servicio Companion 23

5.5. Emparejamiento y transporte privado 24

5.6. Control nativo y control visual del teléfono 25

5.7. Integración de WhatsApp 26

5.8. Acceso desde la web 26

5.9. Expresividad de la interfaz 27

6. Fiabilidad, seguridad y mantenimiento 28

6.1. Control de turnos y recuperación de conexiones 28

6.2. Ruido, interrupciones y naturalidad 29

6.3. Datos privados y permisos 29

6.4. Diagnóstico y mantenimiento 30

II. ELECTRÓNICA — El dispositivo ATLAS A1 **31**
7. Selección de componentes e integración física **31**
7.1. Selección de la Raspberry Pi 31
7.2. Pantalla y entrada táctil 32
7.3. Micrófono, altavoces y Bluetooth 33
7.4. Alimentación, almacenamiento y refrigeración 34
8. ATLAS OS, recuperación y diseño de la carcasa **34**
8.1. Adaptación del sistema operativo 34
8.2. Modos de uso y recuperación local 35
8.3. Diseño digital de la carcasa 36
8.4. Configuración para Bambu Lab H2S 38
8.5. Presupuesto del prototipo 39
8.6. Ampliaciones físicas y criterio de selección 39
III. EVALUACIÓN — Resultados y límites **40**
9. Evaluación del prototipo **40**
9.1. Medición de la experiencia completa 40
9.2. Resultados de las mediciones históricas 40
9.3. Pruebas de voz, rutinas y Android 41
9.4. Baterías de preguntas y mejora de las acciones 42
9.5. Optimización de la rapidez y la calidad 43
9.6. Límites de la evaluación 43
IV. CONCLUSIONES Y CONTINUIDAD DEL PROYECTO **45**
10. Respuesta a la pregunta inicial **45**
11. Aprendizajes de la integración del sistema **45**
12. Desarrollo futuro **46**
Referencias documentales **48**

Introducción

Desde el primer momento, mi idea principal fue crear una inteligencia artificial capaz de hacer algo más que responder preguntas. Quería hablar con ella de forma natural, que recordase información importante y que, cuando le pidiese una tarea, tuviese alguna manera de llevarla a cabo. No me interesaba únicamente construir un chat; quería convertirlo en un agente con un entorno en el que actuar.

Los asistentes de voz fueron una referencia. Me gustaba pedir algo sin abrir una aplicación, pero quería experimentar con un sistema que pudiese adaptar a mis necesidades. A partir de ahí formulé la pregunta principal: ¿era posible construir un agente conversacional, con memoria y herramientas, e integrarlo en un dispositivo físico accesible, manteniendo una interacción suficientemente rápida?

Mi hipótesis fue que sí que era posible, combinando un modelo de lenguaje con software propio y una Raspberry Pi. La investigación se centró en conectar esas piezas, resolver sus limitaciones y comprobar hasta qué punto el resultado podía actuar de forma útil.

Para comprobarlo me propuse construir un flujo de voz funcional, proporcionar contexto persistente y herramientas, crear interfaces para la pantalla, la terminal y Android, e integrar micrófono y altavoces. También debía evaluar la rapidez y la fiabilidad de las acciones, y permitir que una persona pudiese diagnosticar el dispositivo cuando algo fallase.

La metodología fue de desarrollo experimental: preparé una primera solución, observé sus problemas, registré pruebas y fui modificando la arquitectura. Utilicé documentación técnica para entender los componentes y contrasté sus especificaciones con el prototipo. Algunos ensayos comprobaron funciones de software; otros incluyeron dispositivos reales. En la evaluación distingo ambos casos.

Organicé la memoria en cuatro partes: informática, electrónica, evaluación y conclusiones. Los catálogos y registros detallados quedaron en los anexos. La memoria recoge el desarrollo del prototipo y sus resultados; el apartado final reúne las propuestas de continuidad. Mi intención fue explicar un proceso real, con decisiones acertadas y otras que tuve que cambiar.

Objetivos

Mi objetivo principal era crear un dispositivo físico que ejecutase un agente de inteligencia artificial, con su propio entorno de software y herramientas. Quería poder hablar con él de forma natural y que fuese capaz de hacer algo más que responder preguntas: recordar información útil, entender lo que le pedía y actuar sobre su entorno cuando fuese necesario.

Para conseguirlo, me planteé integrar una Raspberry Pi con una pantalla, un micrófono y altavoces. La idea era reunir estos componentes en un prototipo que pudiera utilizar y modificar, sin depender de un ordenador de escritorio para cada interacción. También quería diseñar una carcasa que permitiese organizar las piezas, acceder a los conectores y mantener una ventilación adecuada.

Otro objetivo era preparar un sistema operativo adaptado al dispositivo. No pretendía programar uno desde cero, sino partir de Linux y configurar un entorno propio para ATLAS, con los servicios, las herramientas y la interfaz que necesitase. Del mismo modo, utilizaría un modelo de inteligencia artificial existente: mi trabajo se centraba en integrarlo y darle una forma de interactuar con el sistema.

En la conversación, buscaba que el agente mantuviese el hilo entre varias preguntas y conservase la información importante. También quería añadir herramientas para consultar el estado del dispositivo y realizar tareas, de manera que no tuviese que indicar siempre el nombre de un comando para conseguir algo.

Por último, me propuse comprobar la rapidez, la fiabilidad y la utilidad del prototipo mediante pruebas. Quería registrar los resultados, revisar los errores y comparar los cambios que fuese haciendo, de manera que las mejoras se apoyasen en datos.

Antecedentes

Cuando empecé a plantear ATLAS, la idea de hablar con un dispositivo físico ya estaba presente en muchos hogares. Los altavoces y pantallas inteligentes eran una referencia para el proyecto: permiten hacer preguntas, escuchar música o controlar dispositivos compatibles sin utilizar continuamente un teclado o un móvil. Quería partir de ese concepto, pero construir un prototipo cuyo funcionamiento pudiera desarrollar y modificar yo mismo.

Entre los ejemplos más conocidos está la familia Amazon Echo. Los Echo y Echo Dot integran Alexa en un altavoz, mientras que los Echo Show incorporan una pantalla para complementar la interacción por voz. Esta permite consultar información visual, reproducir contenido y utilizar funciones como las videollamadas. Para ATLAS, el formato con pantalla era especialmente interesante, porque quería que la interacción no dependiera únicamente del sonido (Amazon, 2020).

Google también tiene dispositivos de este tipo. Google Home y los posteriores modelos Nest combinan altavoces, micrófonos y un asistente de voz. Nest Mini mantiene un formato pequeño centrado en el audio, mientras que Nest Hub añade una pantalla para mostrar información, fotografías y controles del hogar. Son dos formas de integrar un asistente en un objeto cotidiano: una más discreta y otra con una interfaz visual (Google, 2019a y 2019b).

Otro antecedente es el HomePod mini de Apple, que incorpora Siri y permite escuchar música, realizar consultas y controlar accesorios domésticos compatibles. También destaca su integración con otros dispositivos y servicios del mismo ecosistema (Apple, 2020).

ATLAS comparte con estos productos la idea de tener un asistente accesible mediante la voz y conectado a herramientas. La diferencia que buscaba era poder decidir cómo se organiza mi propio sistema: trabajar sobre una Raspberry Pi, preparar el entorno basado en Debian, diseñar la interfaz y añadir las herramientas que necesitara. No pretendía superar a todos estos dispositivos ni crear un modelo de inteligencia artificial desde cero, sino comprobar hasta dónde podía llegar integrando modelos existentes en un dispositivo propio.

I. INFORMÁTICA — El agente y su software

1. Concepto del agente conversacional

1.1. Definición de agente

Una conversación por sí sola no permite actuar sobre el mundo. Un modelo puede explicar cómo abrir una aplicación, pero eso no significa que pueda abrirla. Para que lo haga necesita una herramienta: una función externa que reciba una petición, ejecute la operación y devuelva el resultado. Esta diferencia fue la base del proyecto.

En ATLAS utilicé el término agente para referirme al conjunto formado por el modelo, sus instrucciones, el contexto, las herramientas y el programa que coordina todo. El modelo interpreta la petición y propone qué hacer; el software comprueba y ejecuta las operaciones permitidas; después, el resultado vuelve a la conversación. Si una herramienta falla, el agente debe recibir ese fallo, no asumir que la tarea se completó.

Por ejemplo, ante "abre Amazon y busca ESP32", responder "ya está" no demuestra nada. Abrir la aplicación tampoco completa la petición. El sistema necesita continuar hasta introducir la búsqueda y comprobar que aparece el resultado esperado. Esta distinción entre intención, acción y verificación resultó mucho más importante de lo que parecía al comenzar.

La autonomía que buscaba no era ilimitada. Consistía en que ATLAS pudiera escoger pasos intermedios sin obligarme a describir cada pulsación, siempre dentro de los permisos concedidos. No significa que cualquier texto que vea deba convertirse en una orden, ni que deba realizar acciones irreversibles sin tener clara la intención del usuario.

1.2. El nombre y la identidad de ATLAS

Una vez pensada la base del proyecto, tocaba darle un nombre. Quería que fuese corto, fácil de pronunciar y cómodo de repetir. También me gustaba que tuviese algún significado relacionado con su función. Finalmente escogí ATLAS: un nombre que asocié tanto a la idea de sostener como a una colección organizada de información.

Durante la primera etapa utilicé también el nombre OpenAtlas, por la relación con OpenClaw. Más adelante, al construir una arquitectura independiente, el nombre quedó simplemente en ATLAS. ATLAS A1 identifica el dispositivo físico; ATLAS OS, la

adaptación del sistema operativo; y WebScreen, la interfaz que aparece en su pantalla.

Esta separación evita confundir el agente con una sola aplicación. Si hablo con él desde el móvil o desde la terminal, sigue siendo ATLAS. Lo que cambia es el canal de entrada y salida, no la identidad ni el conocimiento principal que recibe.

Busqué una personalidad cercana, con respuestas breves y claras. La cara de la pantalla y sus expresiones acompañan esa intención, pero no representan sentimientos reales. Son recursos de comunicación que ayudan a reconocer cuándo escucha, responde o está esperando. El valor principal del proyecto sigue estando en comprender la petición y actuar correctamente.



Figura 1. Símbolo y nombre de ATLAS sobre una composición gráfica para la web. Identidad propia; fondo elaborado con apoyo de generación de imágenes.

1.3. Desarrollo propio y componentes reutilizados

Para construir el prototipo aproveché tecnologías existentes. El modelo de inteligencia artificial, Linux, el navegador y las bibliotecas de comunicación no son desarrollos míos. Mi trabajo consistió en integrarlos, definir sus responsabilidades y crear las interfaces, herramientas y mecanismos de coordinación específicos de ATLAS.

Esto no eliminó la dificultad técnica. Una función podía funcionar aislada y fallar al combinarla con otra: el micrófono podía abrirse correctamente y, aun así, entrar en conflicto con una segunda sesión; una aplicación podía mantener conexión con la Raspberry Pi mientras la conversación de voz todavía no estaba preparada. Gran parte del desarrollo fue resolver esas relaciones.

Utilicé Codex como apoyo para programar, revisar errores, preparar pruebas y organizar documentación. La ayuda de una herramienta de desarrollo no sustituye la necesidad de entender lo que se integra. Por eso procuré contrastar sus propuestas con los registros y con el comportamiento del dispositivo, y comprobar qué demostraba cada prueba.

Tampoco creé un modelo que aprendiese modificando sus parámetros y pesos cada vez que hablábamos. La adaptación progresiva del prototipo se hizo mediante instrucciones y memoria en archivos. Es una forma de conservar información entre sesiones, pero técnicamente no es un reentrenamiento. Esta precisión es importante para entender qué puede y qué no puede hacer el sistema. Entrenar un modelo de este tamaño desde cero requeriría recursos muy superiores a los disponibles para el proyecto.

2. Evolución de OpenClaw a Realtime

2.1. Base del prototipo inicial

Inicialmente, para la propuesta del prototipo de ATLAS, utilicé OpenClaw, un proyecto para el despliegue y personalización de agentes de inteligencia artificial. La propuesta era buena, fácil de configurar y muy personalizable. Me permitió unir un modelo, instrucciones, memoria y herramientas sin tener que desarrollar toda esa estructura antes de comprobar si la idea tenía sentido.

Durante bastante tiempo fue una parte importante del proyecto. No lo descarté porque no fuese capaz de realizar tareas o porque sus respuestas fuesen necesariamente malas. Al contrario: me ayudó a construir el primer agente funcional y a descubrir qué necesitaba realmente. Su documentación sirvió como referencia para organizar aquella primera etapa. [OpenClaw, documentación general]

El problema apareció cuando comparé ese funcionamiento con la experiencia que quería conseguir. Había funciones y canales instalados que no utilizaba, y la conversación pasaba por demasiadas etapas antes de escucharse una respuesta. La flexibilidad que resultaba cómoda para un prototipo no se traducía automáticamente en una interacción rápida.

Mi objetivo era un agente conversacional con capacidad de actuar, no un sistema que aceptase una orden y permaneciese varios segundos en silencio. Esa diferencia

terminó orientando la evolución de ATLAS más que cualquier característica aislada del modelo.

2.2. El flujo de voz inicial

La primera arquitectura seguía una cadena bastante habitual. El usuario hablaba, el micrófono captaba el sonido y el programa detectaba cuándo terminaba la intervención. Después, un modelo local de transcripción, Whisper, convertía el audio en texto. Ese texto se enviaba al agente gestionado por OpenClaw y, cuando llegaba su respuesta, un motor de voz la pronunciaba.

Cada componente tenía una función razonable, pero dependía del anterior. Si la transcripción no había terminado, el agente no podía empezar; si la respuesta no estaba disponible, el sintetizador tampoco tenía nada que reproducir. Además, la detección del silencio añadía una espera que no se veía en el contenido de la conversación.

En el uso de aquella versión observé esperas habituales de entre 7 y 15 segundos desde el final de mi intervención hasta el inicio de la respuesta hablada. Este intervalo describe mi experiencia con el prototipo y su configuración, no una medida universal de OpenClaw ni un límite de todos los modelos que podía utilizar. Probé distintos modelos de proveedores, pero todos llegaban al mismo cuello de botella, por culpa de la estructura de OpenClaw a la hora de las inferencias.

Ese espacio de silencio se hacía enorme. A veces parecía que ATLAS no había escuchado o que el sistema estaba bloqueado. Aprendí que la calidad de una conversación no depende solo de que la respuesta final sea correcta: también importa lo que ocurre mientras el usuario la espera.

2.3. Transcripción en tiempo real y preámbulos

El primer cambio importante fue sustituir, en la entrada habitual, la transcripción local por el reconocimiento que ofrecía Chrome. De esta manera podía recibir texto mientras hablaba, sin esperar siempre a procesar una grabación completa al terminar. Esto redujo una parte del recorrido, aunque no convirtió el reconocimiento del navegador en una función necesariamente local u offline. [Web Speech API]

También ajusté el criterio para dar por terminada una frase. Un silencio demasiado largo retrasaba todo; uno demasiado corto cortaba una intervención normal en varias partes. No era suficiente bajar un número hasta que pareciese rápido: había que

comprobar qué ocurría con pausas, ruido y frases que continuaban después de una pequeña interrupción.

La segunda ayuda fue lo que denominé "preámbulo". Preparé una ruta más ligera que analizaba la petición y decía algo breve mientras el agente principal seguía trabajando. En ciertas pruebas pude reducir la espera total hacia los 7 u 8 segundos y hacer que hubiese una primera intervención alrededor de los 3 segundos. Esa primera frase no era todavía el resultado de la tarea.

La idea funcionó durante un tiempo, pero introdujo otro problema. Ahora debía coordinar dos respuestas: una que anticipaba lo que se iba a hacer y otra que conocía el resultado real. A veces el preámbulo no se activaba; otras, le faltaba contexto o decía algo que no encajaba del todo. Había reducido el silencio percibido, pero no había eliminado el trabajo que seguía ejecutándose.

2.4. Motivos del cambio de arquitectura

Me costó bastante dejar de lado ese planteamiento. Había invertido tiempo en hacerlo funcionar y cada mejora parecía acercarlo un poco más al resultado que buscaba. Sin embargo, conforme añadía capacidades, también crecían los estados que había que coordinar y los casos en los que la conversación se sentía inconsistente.

En las configuraciones que había probado, incluso las respuestas relativamente sencillas podían tardar cinco o seis segundos antes de añadir las demás etapas. No concluí que ningún modelo pudiera responder más rápido; concluí que mi estructura no estaba resolviendo bien el problema. Seguir afinando una pieza aislada tenía cada vez menos efecto sobre el conjunto.

La alternativa fue utilizar un modelo preparado para conversar en tiempo real: GPT Realtime. Este podía trabajar con audio y texto y solicitar herramientas dentro de la misma sesión. Esto me permitía cambiar la estructura del problema: en lugar de conectar transcriptor, agente y voz como tres bloques obligatoriamente consecutivos, podía mantener una conversación preparada y enviarle la intervención directamente. [OpenAI, conversaciones Realtime]

En turnos sencillos y con la sesión ya abierta, empecé a obtener respuestas del orden de uno a tres segundos. El cambio se notaba mucho más que un ajuste visual o una frase de espera. Aun así, las tareas que necesitaban varias herramientas seguían

dependiendo de su ejecución. Realtime redujo una parte fundamental de la espera; no hizo instantáneo todo lo que ATLAS podía hacer.

2.5. Componentes conservados y retirados de OpenClaw

La migración no consistió simplemente en desinstalar OpenClaw. Antes tuve que separar los datos que pertenecían al proyecto de los servicios que los estaban utilizando. Conservé los archivos de identidad, memoria e instrucciones, revisé sus relaciones y los trasladé a una estructura propia dentro de `.atlas`.

También fue necesario migrar las configuraciones privadas que seguían siendo útiles, como las claves de servicios de voz o búsqueda, sin incluirlas en el repositorio. La autenticación se mantuvo separada del conocimiento del agente. Una clave permite acceder a un servicio; no es una información que deba formar parte del texto que recibe el modelo.

El sistema actual ya no necesita el Gateway ni los canales de OpenClaw para conversar. Sus responsabilidades se repartieron entre el servidor de WebScreen, el componente de autenticación, el motor de herramientas y la memoria propia. Mantener esa distinción evita describir la arquitectura actual como si siguiese siendo una configuración de OpenClaw.

No obstante, esta etapa sigue siendo relevante para la memoria. Fue la base con la que validé la idea y el punto de comparación que justificó el cambio. Retirarlo del producto no significa borrarlo de la historia del proyecto: precisamente el proceso de detectar sus límites y sustituirlo forma parte del trabajo de investigación.

3. Modelos, contexto y herramientas

3.1. Concepto de token

Llegado este punto, considero oportuno explicar algunos conceptos que utilizo a lo largo del documento. El primero es el token. Un modelo no recibe necesariamente el texto dividido palabra por palabra, sino en unidades que pueden ser palabras, fragmentos, signos o espacios según el sistema de tokenización.

Por eso no existe una equivalencia exacta entre palabras y tokens. Como orientación para texto en inglés, OpenAI utiliza la aproximación de cien tokens por unas 75 palabras. En español la proporción puede variar y también cambia con el vocabulario, el código o los números. Para conocer el tamaño real de un texto hay que utilizar el

tokenizador correspondiente, no contar únicamente palabras. [OpenAI, recuento de tokens]

La ficha del modelo GPT Realtime 2.1 indica una ventana de contexto de 128.000 tokens. Si aplicamos solo la aproximación anterior a texto en inglés, obtenemos unas 96.000 palabras: 128.000 x 0,75.

Para poner en contexto (jaja), serían aproximadamente 768 textos de 125 palabras que ATLAS podría tener disponibles dentro de una conversación. Esto daba un margen de trabajo espectacular para el proyecto. [OpenAI, GPT Realtime 2.1]

Tampoco significa que todo ese espacio esté reservado a los documentos del usuario. Las instrucciones, la conversación y los resultados de las herramientas también forman parte de lo que el sistema debe manejar. En una sesión con audio, además, no se debe convertir sin más todo su consumo en un número de palabras escritas.

3.2. Contexto y límites de memoria

El contexto es la información disponible para el modelo dentro de una interacción. Una comparación útil es imaginar una mesa de trabajo: cuanto más grande sea, más documentos se pueden tener abiertos a la vez. Pero tener un documento sobre la mesa no garantiza interpretar correctamente cada detalle ni recordar siempre dónde estaba.

Esta distinción corrigió una idea demasiado optimista que había planteado al principio: una ventana grande no equivale a un cerebro que conserva cada palabra sin equivocarse. El modelo puede confundir información, no utilizar un dato relevante o responder con más seguridad de la que debería. Por eso las tareas importantes necesitan herramientas y comprobaciones, incluso cuando la información cabe en el contexto.

Para ATLAS, la capacidad disponible permitió incluir bastante información sobre el usuario, el dispositivo y las funciones del proyecto. Sin embargo, no quise llenarla por el simple hecho de poder hacerlo. Repetir una misma instrucción en varios archivos aumenta el tamaño y puede crear contradicciones cuando se actualiza solo una copia.

La solución fue asignar responsabilidades a los documentos: uno explica la identidad, otro la personalidad, otro los datos del proyecto y otros las herramientas. Un manifiesto organiza qué se carga. Esta estructura resulta más fácil de mantener que una única explicación enorme donde todo se mezcla, y heredé la idea principal de OpenClaw.

3.3. Memoria entre sesiones

La memoria persistente resuelve un problema distinto. Al cerrar una conversación, no basta con confiar en que el modelo recordará lo hablado en la siguiente. Si un dato debe conservarse, tiene que guardarse en un soporte que el sistema pueda recuperar después.

En ATLAS separé el conocimiento relativamente estable de la información conversacional que cambia. El primero contiene instrucciones y manuales. La segunda recoge estado y recuerdos que pueden actualizarse durante el uso. Así pude revisar los documentos del proyecto sin tratarlos como si fuesen un registro de todos los turnos anteriores.

Por ejemplo, una preferencia del usuario puede ser relevante para próximas conversaciones; la salida completa de una prueba temporal o el output de un comando cualquiera, probablemente no. Guardar indiscriminadamente todo no solo ocupa espacio: también hace más difícil encontrar lo útil y aumenta la cantidad de información personal almacenada.

Este mecanismo permite adaptación progresiva, pero no cambia los parámetros del modelo. Lo que se modifica es la información que ATLAS consulta o recibe. Mantener esta diferencia ayuda a describir el proyecto con precisión y a no atribuirle una forma de aprendizaje que no se ha implementado.

3.4. Herramientas y resultados verificables

Una herramienta es una interfaz concreta entre el modelo y una capacidad externa. Puede consultar el estado de la Raspberry Pi, abrir una aplicación o ejecutar una rutina. Su contrato define qué datos necesita, qué devuelve y qué errores pueden aparecer.

Intenté que las operaciones habituales tuviesen herramientas específicas. "Consultar ubicación" resulta más claro que entregar una shell y esperar que el agente descubra desde cero cómo hablar con Android. Una función estructurada permite validar parámetros, distinguir una falta de permiso de una desconexión y evitar interpretar cualquier salida como un éxito.

El resultado vuelve a la conversación para que ATLAS pueda continuar. Si abre una aplicación y la petición exige actuar dentro de ella, necesita una segunda capacidad: observar e interactuar con su interfaz. Si una rutina obtiene un dato simple, puede no

necesitar más razonamiento. La herramienta adecuada depende de la tarea, no de que el agente quiera utilizar siempre la opción más compleja.

La autenticación de un servicio tampoco concede por sí sola todas las capacidades de una cuenta. Disponer de una sesión válida no hace aparecer automáticamente Gmail, Drive u otras integraciones en Realtime. Cada acceso necesita una herramienta realmente conectada y una autorización válida, junto a una adaptación que actue de puente entre esta y el agente. Esta fue otra separación necesaria entre lo que un modelo podría hacer y lo que ATLAS tenía implementado.

4. Arquitectura propia de ATLAS

4.1. Componentes y distribución de funciones

Después de la migración organicé ATLAS como un conjunto de componentes pequeños. El navegador se ocupa de la interacción visible y del audio; el servidor prepara sesiones y herramientas; el sistema de autenticación mantiene el acceso al proveedor; y los servicios del dispositivo ejecutan las operaciones que les corresponden. Aunque el usuario perciba un solo agente, internamente hay varias responsabilidades.

Esta separación permitió diagnosticar mejor los fallos. Una pantalla abierta no demuestra que Realtime esté conectado; una sesión Realtime activa no demuestra que el teléfono esté disponible. Antes, esos estados podían mezclarse en un mensaje genérico de "conectando", que resultaba poco útil tanto para el usuario como para mí.

La Raspberry Pi actúa como punto de coordinación. Allí se encuentran el contexto, las rutinas, el servidor local y los servicios del A1. El modelo conversacional se ejecuta en la nube. Por tanto, el proyecto utiliza una arquitectura híbrida: parte del trabajo es local y otra parte depende de servicios externos. No sería correcto presentarlo como un asistente completamente offline.

Una ventaja de esta organización es que una interfaz puede cambiar sin tener que rehacer toda la lógica. La pantalla minimalista y la vista de depuración, por ejemplo, reutilizan el mismo flujo de conversación. No quise mantener dos agentes que se comportasen de forma distinta por utilizar diseños diferentes. Las responsabilidades de cada componente se pueden consultar en el anexo A, apartado A.1.

4.2. Native Broker y preparación de las sesiones

Llamé Native Broker al componente que concentra la relación con la autenticación y la preparación de Realtime. En la implementación del prototipo utiliza el servidor de aplicación de Codex y conserva separado su estado de acceso. De esta forma, el navegador no necesita recibir la credencial persistente de la cuenta.

Esta ruta de acceso se utilizó en mi entorno de desarrollo, pero no supone que una suscripción de ChatGPT o Codex conceda, de forma general, uso ilimitado de cualquier API. La disponibilidad, las cuotas y las condiciones dependen del servicio contratado. La distribución del producto requiere una vía de acceso soportada y apropiada para cada usuario.

También aprendí a separar errores de autenticación de errores de transporte o del proveedor. Un mensaje de servidor 500 no demuestra que se haya corrompido el inicio de sesión. Volver a autenticar puede ser necesario en algunos casos, pero hacerlo sin comprobar los registros solo cambia una pieza y puede dejar intacto el problema real.

Preparar una sesión tiene un coste diferente al de responder dentro de una ya preparada. Se negocia la conexión, se configura la voz y se carga el contexto. Por eso procuré no repetir ese proceso en cada pulsación del micrófono. El objetivo era que la conversación estuviese disponible al necesitarla, sin mantener recursos costosos activos cuando no aportaban nada.

4.3. Organización de archivos y contexto

La carpeta `.atlas` reúne el software y los datos del proyecto, pero no todo se guarda mezclado. Dentro de `context/knowledge` se encuentran los documentos de conocimiento e instrucciones. En `context/conversation` se conserva el estado conversacional mutable. Los servicios, las rutinas, los perfiles y la configuración privada tienen espacios propios.

Los nombres de algunos documentos hacen explícita su responsabilidad. `IDENTITY.md` describe quién es ATLAS; `SOUL.md`, su comportamiento; `USER.md`, información relevante del usuario; y `TOOLS.md`, una guía de capacidades. `AGENTS.md` relaciona las distintas piezas. Un manifiesto establece el orden de carga y evita incorporar dos veces un mismo archivo.

Para el trabajo de investigación apliqué el mismo criterio. `TDR.md` contiene los datos y plazos particulares del proyecto. `TDRMAN.md` resume las normas del instituto. La

transcripción completa del manual se conserva aparte y se consulta cuando hace falta, sin añadir todas sus páginas a cada turno de conversación.

La estructura no pretende ser una memoria ilimitada. Es una forma controlable de decidir qué información recibe el agente. La recuperación selectiva de documentos evita cargar información innecesaria y permite comprobar que no se pierde lo importante. Añadir archivos sin revisar su contenido no es una estrategia suficiente de gestión del contexto.

4.4. Rutinas para tareas repetitivas

No todas las tareas necesitan que el modelo razone de nuevo. Si le pido la hora, el sistema puede ejecutar una consulta local y convertir el resultado en una frase. Hacer que un agente explore varias herramientas para resolver eso añade tiempo y oportunidades de error sin aportar una ventaja real.

Para estas situaciones preparé un sistema de rutinas. Cada rutina tiene un nombre, una o varias frases de activación y pasos definidos. Puede ejecutar un comando, guardar su salida en una variable y utilizarla en una respuesta. El registro incluye si el resultado necesita interpretación del modelo o si puede devolverse directamente.

En una rutina de hora, por ejemplo, no se guarda una respuesta fija como "son las cinco". Se guarda cómo obtener la hora actual y cómo insertarla en la frase. Así, la ejecución es repetible sin dejar de depender de un dato real del sistema. La voz todavía necesita sintetizar esa frase, por lo que ejecutar el comando en milisegundos no garantiza oírla en ese mismo tiempo.

La creación o modificación de una rutina sí puede requerir al agente. Primero hay que entender qué quiere el usuario, identificar el dispositivo correcto y comprobar que el procedimiento funciona. Una vez resuelta esa parte, el uso cotidiano puede seguir una ruta más directa. Si falla, se devuelve el error para analizarlo sin repetir a ciegas una acción con efectos.

4.5. Perfiles y límites de capacidades

También preparé una estructura declarativa de perfiles, a los que llamé roles. La idea es que no todas las instalaciones necesiten el mismo conocimiento ni las mismas herramientas. Un dispositivo de consulta escolar, por ejemplo, no debería recibir acceso administrativo solo porque el prototipo personal lo tenga.

El perfil habitual es atlas-full. Incluye las capacidades completas del prototipo y, para las pruebas, información de profesores. También preparé manifiestos de perfiles más limitados para estudiar otros usos. Un manifiesto describe las capacidades de un perfil, pero su aislamiento depende de los controles aplicados por software.

Esta distinción es importante porque una instrucción escrita no es una barrera de seguridad. Indicar "no utilices la shell" en un documento puede orientar al modelo, pero la protección real consiste en no ofrecer esa herramienta o rechazar su ejecución desde el programa que la controla.

El ejemplo de profesores permitió estudiar cómo responder a partir de un contexto acotado y de la hora actual, sin convertir una consulta sencilla en una investigación larga. La información de profesores y horarios se utilizó como contexto de prueba, no como un despliegue oficial del instituto.

5. Interfaces y conexión con otros dispositivos

5.1. WebScreen y conversación por voz

WebScreen es la página que muestra ATLAS A1. Utiliza HTML, CSS y JavaScript para la interfaz y un servidor local para las funciones del dispositivo. La voz en tiempo real se transporta mediante WebRTC, mientras que otros mensajes de control y resultados de herramientas se coordinan con el servidor.

La palabra de activación es "ATLAS". El controlador decide cuándo una intervención puede iniciar un turno y cuándo debe ignorarse. Esta decisión no puede depender solo de pedir al modelo que espere: si ya se ha abierto una respuesta, el problema de coordinación ha ocurrido antes de que la instrucción de personalidad pueda resolverlo.

Durante el desarrollo probé mantener una ventana de escucha después de responder. En una primera revisión la retiré para evitar que ATLAS incorporase frases dirigidas a otra persona. Más adelante añadí una ventana de 3 segundos: al terminar de hablar, sigue escuchando y permite continuar sin repetir su nombre. Si comienzo una frase dentro de ese intervalo, me deja terminarla; si no, vuelve a esperar la palabra de activación. El tiempo cuenta desde el final real del audio.

La interfaz tiene una vista técnica con registros y otra minimalista para el uso diario. Las expresiones y animaciones indican estados y hacen al agente más cercano; no modifican por sí mismas el control de la conversación. También incorporé ajustes

locales de red y un teclado táctil llamado "teclao" (me parecía gracioso el nombre jaja), para poder conectar la Raspberry Pi a otro Wi-Fi desde su pantalla.

La página local puede mostrarse aunque no haya Internet, siempre que el servidor y el navegador del A1 funcionen. Eso no significa que el modelo o todos los servicios de voz funcionen sin red. Distinguir ambas cosas es esencial: una interfaz disponible puede servir para recuperar la conexión, aunque la conversación no esté preparada.



Figura 2. Prototipo real con pantalla y micrófono. Fotografía propia retocada para eliminar reflejo y polvo y sustituir el reloj por la cara habitual.

5.2. Voz nativa y motores alternativos

Con Realtime, la ruta de voz nativa permite que el modelo produzca audio como parte de su respuesta. Ya no es obligatorio esperar a un texto completo, enviarlo a otro servicio y recibir después un archivo de voz. Esta fue una de las razones principales para cambiar de arquitectura.

Sin embargo, conservé opciones como ElevenLabs y la síntesis del navegador. Podían interesarme por su timbre o por la disponibilidad de una voz concreta. En esos casos sí existe una etapa adicional de síntesis. Decir que Realtime eliminó toda transcripción y todo motor externo en cualquier configuración sería simplificar demasiado: eliminó la necesidad de esa cadena en la ruta nativa.

Para reducir esperas trabajé con respuestas progresivas o streaming. En lugar de esperar al resultado completo, el programa recibe fragmentos y empieza a procesarlos cuando tiene información suficiente. Eso exige cuidar los límites de las frases, la

reproducción y las cancelaciones; no basta con dividir arbitrariamente un texto en trozos.

La documentación de ElevenLabs distingue diferentes factores de latencia, y la medición del prototipo confirmó que había que observar más de un instante. Recibir el primer byte de audio es distinto de que el navegador pueda reproducirlo, y también de escucharlo por el altavoz. Esta diferencia aparece de nuevo en la evaluación. [ElevenLabs, optimización de latencia]

5.3. El chat de terminal como herramienta de pruebas

Creé `atlas-chat` para poder interactuar por texto con la misma base conversacional y de herramientas. Me permitía repetir una petición sin depender del micrófono, comprobar qué comando había solicitado el modelo y ver su salida. Así podía separar un error de lógica de un problema de audio o reconocimiento.

El modo interactivo mantiene una conversación y el parámetro `-p` permite enviar una petición desde la línea de comandos. La interfaz diferencia las respuestas del agente de la actividad técnica, e incluye historial, edición de entrada y sugerencias. Son ayudas para depurar, no capacidades nuevas del modelo.

También dispone de un modo de prueba que no incorpora los turnos a la memoria conversacional compartida. Esto evita que una serie de ensayos artificiales se convierta en recuerdos habituales del usuario. No debe confundirse con una garantía de ausencia de registros: un diagnóstico puede seguir dejando trazas técnicas que necesitan su propia política de conservación.

Una prueba satisfactoria en terminal demuestra que cierto recorrido de texto y herramientas funciona. No demuestra que el micrófono detecte bien la palabra ATLAS, que no haya eco o que el altavoz reproduzca la respuesta. La terminal resultó útil precisamente porque permite aislar esa parte del problema.

5.4. Aplicación Android y servicio Companion

La aplicación Android amplió el acceso al agente fuera de la pantalla del A1. Organicé sus funciones en conversación, acciones, terminal, estado y ajustes. El objetivo era poder hablar con ATLAS, ejecutar acciones preparadas y consultar el dispositivo desde el teléfono, manteniendo una experiencia coherente.

En la Raspberry Pi, el servicio Companion gestiona la relación con la aplicación. La conexión de control y la sesión de voz son distintas: mantener el teléfono vinculado no obliga a mantener el micrófono ni WebRTC abiertos todo el tiempo. Esta separación fue especialmente importante al optimizar el consumo en segundo plano.

El enlace de control utiliza un servicio visible para Android y puede mantenerse al minimizar la aplicación, mientras el sistema operativo permita ejecutar el proceso. No es correcto garantizar una conexión eterna: Android, el fabricante, una pérdida de red o una orden de forzar detención pueden interrumpirlo. La aplicación debe reconocer el estado y recuperarse, no fingir que sigue conectada.

La terminal también se gestiona como una sesión propia. Puede sobrevivir a una recreación de la pantalla de la aplicación y cerrarse tras un período de inactividad. En el estado del dispositivo conservé el último dato válido mientras se solicitaba uno nuevo, para que una recarga no dejase la interfaz vacía. Mostrar un dato anterior exige distinguirlo de una lectura reciente cuando la conexión falla.

Los permisos se solicitan explicando su finalidad. Algunos son necesarios para una función concreta y otros son opcionales. Dar permiso al micrófono no autoriza a acceder a contactos; conceder accesibilidad tampoco significa que el agente deba utilizarla para cualquier operación. La interfaz de permisos forma parte del control que conserva el usuario.

5.5. Emparejamiento y transporte privado

Inicialmente probé una conexión basada en un relay externo. Más adelante pasé a Tailscale para conectar el portátil, la Raspberry Pi y el teléfono dentro de una red privada. Esta decisión buscaba simplificar el acceso entre mis dispositivos y favorecer una ruta directa cuando fuese posible.

Tailscale intenta establecer conexiones directas y puede recurrir a un relay cuando las condiciones de la red lo impiden. Estar en la misma Wi-Fi suele favorecer el enlace directo, pero no conviene asumirlo sin comprobarlo. Un túnel privado tampoco elimina la latencia de los servidores del modelo ni la de una captura de pantalla. [Tailscale, tipos de conexión]

Mantuve un emparejamiento propio de ATLAS. El A1 abre una ventana temporal y muestra un código de seis cifras; el teléfono lo introduce durante el proceso de

vinculación. El código facilita una entrega autorizada de credenciales, pero no sustituye la clave que protegerá las sesiones posteriores.

La pertenencia a la red privada y el emparejamiento son capas diferentes. Que un equipo alcance la dirección del A1 no significa que deba poder controlarlo. El transporte y los mensajes se protegen con mecanismos de cifrado, y el desemparejamiento invalida la relación para que no baste con conservar una pantalla antigua de la aplicación.

5.6. Control nativo y control visual del teléfono

Dividí el control del teléfono en dos recorridos. El primero utiliza funciones nativas: abrir una aplicación o consultar información autorizada mediante una operación definida. El segundo, Android Use, interactúa con la pantalla cuando no existe una función directa adecuada.

Esta diferencia evita utilizar capturas y coordenadas para todo. Si el sistema sabe abrir una aplicación por su identificador, no necesita recorrer visualmente el lanzador. Pero si la petición continúa dentro de esa aplicación, puede ser necesario tomar una captura, leer elementos accesibles y realizar gestos. Abrir la aplicación no debe confundirse con terminar toda la tarea.

Android Use sigue un ciclo de observación, acción y comprobación. Primero obtiene información de la pantalla; después ejecuta una acción o un pequeño lote de acciones previsibles; finalmente comprueba el nuevo estado. El lote reduce intercambios innecesarios, pero no debe ser tan largo que continúe actuando sobre una pantalla diferente de la esperada.

Cuando hay etiquetas accesibles fiables, se puede pulsar un elemento por su descripción. Cuando no las hay, las coordenadas siguen siendo útiles. Ambas estrategias necesitan una observación reciente: una captura antigua no garantiza que el botón continúe en el mismo lugar después de cargar contenido, abrir el teclado o cambiar de orientación.

El servicio de accesibilidad permite las operaciones correspondientes dentro de los límites de Android y debe ser habilitado por el usuario. Mientras ATLAS controla visualmente el teléfono se muestra una indicación y una forma clara de detenerlo. No se intentan eludir las pantallas protegidas ni convertir el permiso en un acceso oculto. [Android Developers, AccessibilityService]

Las pruebas de consultas y navegación no equivalen a validar llamadas o mensajes reales. Esas acciones tienen efectos sobre otras personas y requieren condiciones de ensayo distintas. En la documentación separé las capacidades implementadas, los permisos disponibles y las operaciones efectivamente comprobadas. Las comprobaciones realizadas con Android se recogen en el anexo B, apartado B.5.

5.7. Integración de WhatsApp

Para utilizar ATLAS desde WhatsApp integré Baileys y preparé un servicio propio en la Pi. Permite vincular mi cuenta con un QR, consultar mensajes, responder y programar tareas. Separé dos modos: Guardian, para hablar con ATLAS, y Sami absent, para responder en las conversaciones que yo selecciono.

Guardian responde cuando escribo "@atlas" desde mi cuenta. Recibe el nombre y el tipo de chat, hasta 30 mensajes recientes y, si estoy respondiendo a alguien, el mensaje citado. Después solo se añaden los mensajes que todavía no ha recibido. Mantiene la sesión preparada y conserva las herramientas de ATLAS, de modo que puedo pedirle acciones como en el chat del dispositivo.

En Sami absent preparé un perfil de mi forma de escribir y un índice privado de ejemplos. No entrené un modelo nuevo: utilicé instrucciones y una búsqueda de ejemplos. Como este modo recibe mensajes de otras personas, no tiene shell ni acceso general al sistema; solo puede consultar ese índice de lectura. Puede responder en varias burbujas, citar mensajes o no contestar cuando no hace falta.

También añadí recordatorios con un mensaje fijo o una instrucción para el modelo, como resumir un grupo. El servicio controla el destinatario y el horario. Un monitor muestra las acciones y sus tiempos para localizar fallos. Las credenciales y conversaciones personales no se publican en GitHub. La conexión, los dos modos de conversación y las automatizaciones se explican en el anexo E, apartados E.1 a E.4.

5.8. Acceso desde la web

La web atlas.samilososami.com presenta el proyecto. También incluye dos accesos protegidos con contraseña: "/try", para probar la interfaz del WebScreen, y "/chat", para conversar por escrito. El chat permite guardar varias conversaciones en el navegador, adjuntar imágenes y ver la respuesta mientras se genera. Ambos accesos comparten la sesión, por lo que no hace falta introducir la contraseña dos veces.

El portal funciona como un servicio Python en la Raspberry Pi, accesible mediante HTTPS y Tailscale Funnel. La contraseña se guarda como un hash con sal y el navegador conserva una cookie de sesión revocable. Esta protección era necesaria porque el portal da acceso a las capacidades reales de ATLAS. El historial se guarda en el navegador, aunque los mensajes necesarios para responder se envían a la Pi y al proveedor del modelo.

Preparé instrucciones específicas para la conversación escrita. Por ejemplo, el chat mostraba "48,3 °C" en lugar de escribir el número completo con palabras, como ocurría con la voz. Los avisos de herramientas aparecen junto a la respuesta. Cada chat tiene su propio historial, separado de la pantalla física y de WhatsApp. Los detalles de autenticación, historial y separación de sesiones se pueden consultar en el anexo E, apartados E.5 y E.6.

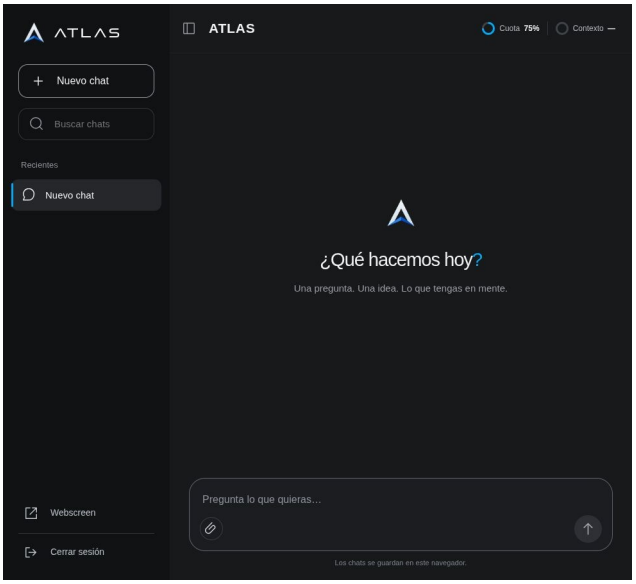


Figura 3. Interfaz real de /chat, sin conversaciones personales. Captura propia del portal, 18 de octubre de 2026.

5.9. Expresividad de la interfaz

A partir de los ojos azules y la sonrisa de ATLAS preparé un catálogo de 323 expresiones: las 23 que ya tenía y dos colecciones nuevas de 150. Combiné párpados, cejas, bocas y algunos accesorios, manteniendo las proporciones y el grosor de los trazos. Las imágenes generadas sirvieron como referencia; la interfaz dibuja las caras mediante vectores.

Las expresiones tienen transiciones de entrada y salida y comparten siete gestos básicos. Por tanto, son 323 poses, no 323 animaciones independientes. Limité los

cambios automáticos a dos por respuesta y los coordiné con la voz para que ATLAS pudiera expresarse sin cambiar de cara constantemente. El catálogo y su funcionamiento se amplían en el anexo F, apartado F.1.



Figura 4. Tres expresiones de ATLAS: neutral, enfadado y triste. Capturas del renderizador vectorial. Elaboración propia.

6. Fiabilidad, seguridad y mantenimiento

6.1. Control de turnos y recuperación de conexiones

Una parte considerable del trabajo fue conseguir que una conversación no se rompiese ante eventos pequeños. El navegador podía notificar un cambio temporal de conexión, llegar un fragmento tardío de transcripción o completarse una respuesta justo cuando se intentaba cancelarla. Tratar todos esos casos como errores graves provocaba reconexiones innecesarias.

Por eso separé el estado de transporte del estado del turno. No es lo mismo que la red esté momentáneamente inestable, que la sesión haya caducado o que un usuario interrumpa una respuesta. Cada situación necesita una reacción distinta. Reiniciar todo indiscriminadamente puede borrar el contexto temporal de una intervención que todavía estaba en curso.

También fue necesario relacionar los eventos con su intervención concreta. Una señal de que terminó un audio anterior no debe cerrar el turno que acaba de empezar. De la misma manera, pedir una cancelación cuando ya no hay respuesta activa no debería provocar por sí solo una nueva conexión completa.

La recuperación mejora la experiencia, pero no puede mantener una conexión física que ya no existe. Ante una interrupción real, el objetivo es detectar el fallo, conservar lo recuperable y restablecer el servicio sin repetir acciones con efectos. La interfaz no debe prometer que no hubo corte si se perdió parte de una petición.

6.2. Ruido, interrupciones y naturalidad

Un agente de voz recibe mucho más que frases limpias. Hay ruido de fondo, conversaciones cercanas, el sonido de su propio altavoz y pausas dentro de una intervención. Si cualquier variación de volumen cambia el estado entre escuchar y pensar, la interfaz parece indecisa y el modelo puede recibir fragmentos solapados.

Trabajé en umbrales de entrada y en la estabilidad de esos estados, buscando un equilibrio entre sensibilidad y falsas activaciones. Una voz muy baja no tiene por qué iniciar el mismo recorrido que una petición dirigida claramente al dispositivo. Sin embargo, un umbral fijo demasiado alto también puede excluir a un usuario que habla suavemente o está más lejos. La calibración tiene límites y necesita pruebas acústicas reales.

La naturalidad tampoco se resolvía añadiendo frases supuestamente humanas. Responder "voy a consultar mi contexto" a una pregunta sencilla explicaba un proceso interno que el usuario no necesitaba escuchar. Preferí instrucciones para responder de forma breve, reconocer lo que faltaba y evitar preámbulos automáticos cuando el dato podía darse directamente.

Una expresión animada puede acompañar esa respuesta, pero no debe controlar cuándo se admite voz ni cuánto tarda en ejecutarse una herramienta. Mantener la presentación separada del controlador fue importante para poder mejorar la apariencia sin introducir cortes en la conversación.

6.3. Datos privados y permisos

ATLAS maneja información personal y herramientas que pueden modificar dispositivos. Las credenciales de servicios, los datos de emparejamiento y el contexto del usuario no deben tratarse como archivos públicos del proyecto. Separé esos elementos de los materiales que se pueden versionar y documentar.

Las claves se guardan con acceso restringido y no deberían aparecer en capturas, salidas de comandos ni imágenes distribuibles. Los registros también necesitan cuidado: pueden incluir transcripciones o información de una pantalla. Que sean útiles para depurar no justifica conservarlos indefinidamente o subirlos completos a un repositorio.

Las instrucciones de prudencia ayudan, pero no sustituyen controles técnicos. Una función puede limitar qué operación acepta, requerir permiso o devolver un error si el

teléfono no está disponible. Las acciones destructivas necesitan especial atención a su objetivo y reversibilidad. El modelo no debe interpretar un contenido encontrado en una página como una nueva autorización del propietario.

La utilidad del agente aumenta con sus capacidades, pero también lo hace la responsabilidad de limitar su alcance.

6.4. Diagnóstico y mantenimiento

Preparé comandos para revisar el estado general, la conexión de la aplicación y los servicios. atlas-rafas concentra comprobaciones y operaciones de diagnóstico; atlas-status resume el sistema; y los registros permiten reconstruir una secuencia de eventos. El catálogo abreviado de comandos se puede consultar en el anexo A, apartado A.2.

Así pude distinguir fallos de red, de audio o de Companion sin atribuirlos todos al modelo. Utilicé systemd para gestionar los servicios, Git para conservar cambios y GitHub para distribuir las versiones. Poder volver a una versión conocida resultó tan importante como introducir funciones nuevas.

No todos los fallos deben repararse automáticamente. Reiniciar el equipo, cambiar de red o modificar credenciales puede interrumpir trabajo válido. El diagnóstico debe distinguir entre observar y modificar, y conservar información suficiente para entender qué ocurrió.

II. ELECTRÓNICA — El dispositivo ATLAS A1

7. Selección de componentes e integración física

7.1. Selección de la Raspberry Pi

Hasta este punto, ATLAS podía entenderse como un conjunto de programas. Sin embargo, mi idea inicial incluía un dispositivo dedicado: algo que pudiese estar en una mesa, escuchar y responder sin depender de mantener abierto mi portátil. ATLAS A1 fue la primera materialización de esa idea.

Escogí una Raspberry Pi 5 con 4 GB de memoria RAM. Dispone de un procesador ARM de cuatro núcleos a 2,4 GHz y conexiones suficientes para pantalla, audio y red. Su tamaño y la documentación disponible facilitaron el montaje de un prototipo ampliable. Las características técnicas de referencia proceden del fabricante; el uso que les di corresponde al desarrollo de ATLAS. [Raspberry Pi, Raspberry Pi 5]

La elección fue un compromiso. Un ordenador más potente habría dado más margen para modelos locales, pero también habría cambiado el tamaño, el consumo y el coste del dispositivo. Un microcontrolador habría sido apropiado para leer sensores o controlar salidas sencillas, pero no para ejecutar cómodamente el navegador y todo el entorno de ATLAS.

En este diseño, la Raspberry Pi no ejecuta el modelo conversacional grande. Coordina servicios locales y se comunica con el proveedor en la nube. Esto permite utilizar un ordenador pequeño, a cambio de depender de la conectividad y del servicio externo. La elección del hardware y la arquitectura de software no fueron decisiones independientes.

Los cuatro gigabytes tampoco son un espacio exclusivo para ATLAS. Se comparten entre sistema operativo, navegador, audio y procesos auxiliares. Por eso tuve que prestar atención al número de sesiones, a los recursos en segundo plano y a qué servicios merecía la pena mantener activos.

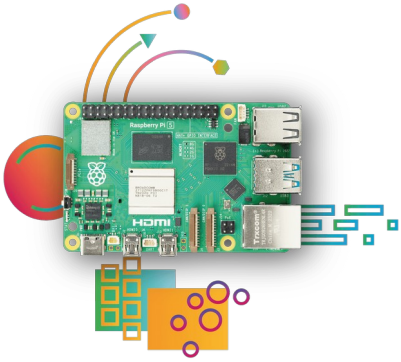


Figura 5. Raspberry Pi 5, placa utilizada como base del A1. Fuente: Raspberry Pi, ficha oficial del producto.

7.2. Pantalla y entrada táctil

Utilicé una pantalla SunFounder TS-7 Pro de siete pulgadas, con resolución de 1024 × 600 píxeles. Recibe la imagen por HDMI y comunica el tacto por USB; también incorpora altavoces. Tener varias funciones en un mismo conjunto redujo la cantidad de módulos que necesitaba añadir al prototipo. [SunFounder, Hardware introduction]

La pantalla condicionó la interfaz. En siete pulgadas, llenar el espacio de indicadores y registros puede ser útil para depurar, pero no para hablar con un agente. Por eso mantuve la vista técnica y preparé otra mucho más sencilla para el uso normal. El diseño debía seguir siendo legible a cierta distancia y permitir reconocer el estado sin estudiar un panel.

Durante las primeras pruebas apareció un caso que ayudó a separar hardware y software: se veía el arranque y después la pantalla mostraba que no recibía señal. Eso no significaba necesariamente que el cable o el panel estuviesen averiados. El sistema necesitaba mantener una sesión gráfica apropiada después de la fase inicial.

Otra dependencia apareció con el modo oculto. Apagar físicamente la pantalla podía hacer desaparecer también la salida de audio que utilizaban sus altavoces. Para poder seguir hablando con ATLAS sin mostrar la interfaz, preparé un modo de pantalla negra y brillo mínimo que mantiene el enlace necesario. Es una ocultación visual, no un apagado eléctrico completo.

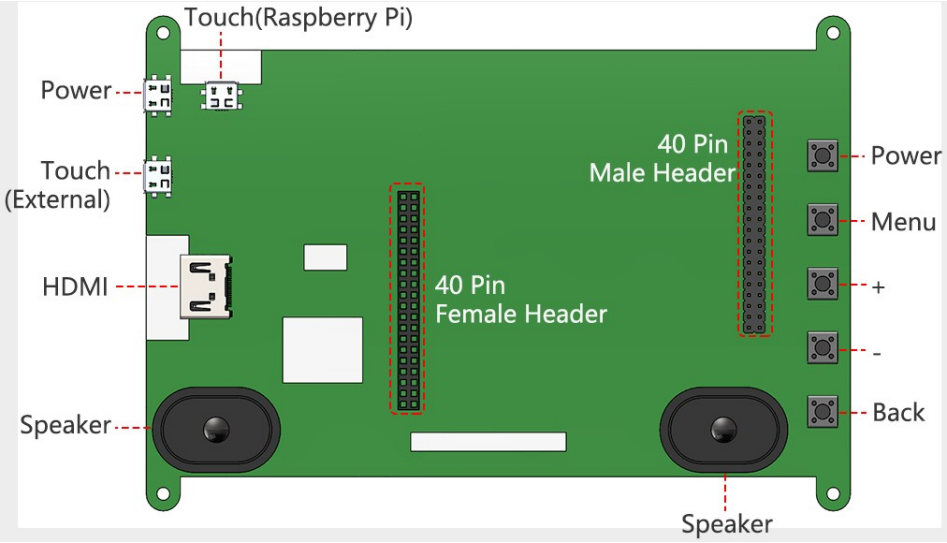


Figura 6. Conexiones, altavoces y cuatro anclajes de la pantalla TS7 Pro. Fuente: SunFounder, Hardware Introduction.

7.3. Micrófono, altavoces y Bluetooth

La entrada de voz del prototipo utiliza un micrófono USB que Linux identifica como Jieli UACDemoV1.0. Trabaja con un canal mono y una frecuencia de muestreo de 48 kHz. El sistema puede detectar su conexión como la de cualquier otro periférico USB, pero reconocer el dispositivo no demuestra que el navegador esté capturando correctamente.

La colocación del micrófono y de los altavoces influye directamente en la conversación. Si ATLAS oye su propia voz, puede confundirla con una nueva intervención. Las protecciones de software ayudan, pero no sustituyen un buen diseño acústico. Por este motivo separé las entradas y salidas de audio y traté el retorno acústico como una limitación específica del prototipo.

También probé una barra de sonido Bluetooth. En ese proceso comprobé que emparejar y reproducir audio son operaciones distintas. El dispositivo podía figurar como conocido y, sin embargo, no disponer del perfil adecuado para la reproducción. Fue necesario revisar la relación entre BlueZ, el gestor de audio y la sesión de usuario.

PipeWire y WirePlumber intervienen en la selección de destinos de sonido. La verificación de la salida combina la selección del destino y el volumen con la comprobación del altavoz real. Un tono de prueba oído físicamente proporciona una evidencia diferente de una línea en el registro.

Estas pruebas mostraron que el tiempo de respuesta no depende solo de la inferencia del modelo. Si el audio se envía a un destino incorrecto, la conversación parece no

responder aunque el texto ya esté completo. El diagnóstico debe recorrer toda la cadena, desde la captura hasta la reproducción.

7.4. Alimentación, almacenamiento y refrigeración

La alimentación tenía que sostener la Raspberry Pi y los periféricos conectados. El fabricante recomienda una fuente USB-C adecuada para la Pi 5, capaz de suministrar 5 V y 5 A. Esta referencia no permite deducir por sí sola el consumo constante del montaje: la demanda cambia según la carga y los dispositivos utilizados. [Raspberry Pi, Raspberry Pi 5]

Durante parte del desarrollo utilicé alimentación portátil. Cuando una batería se agotaba, el equipo podía apagarse de forma brusca y después parecer un fallo del programa. Añadir información sobre arranques y apagados ayudó a distinguir una orden normal de apagado de una interrupción inesperada.

La refrigeración se realizó mediante un Active Cooler. Mantener la temperatura controlada es importante tanto para el hardware como para interpretar las pruebas: si el procesador reduce su frecuencia por calor, cambia el rendimiento aunque el código sea el mismo. El diseño de la carcasa incorpora rejillas de ventilación para facilitar el flujo de aire.

El almacenamiento contiene el sistema, las aplicaciones, los archivos de contexto y los registros. Estos últimos pueden crecer durante la depuración y necesitan una política de conservación. También hay que distinguir la capacidad nominal del soporte del espacio disponible en una partición: el sistema de archivos y los datos existentes reducen lo que queda libre.

Para las actualizaciones habituales es más razonable distribuir los archivos modificados que generar una copia completa del disco. Una imagen del sistema es útil para instalar una base conocida, pero ocupa más, tarda en prepararse y exige revisar que no incluya credenciales, redes privadas o datos personales del prototipo.

8. ATLAS OS, recuperación y diseño de la carcasa

8.1. Adaptación del sistema operativo

Llamé ATLAS OS a la adaptación de Linux que preparé para el A1. Está basada en el entorno Debian y Raspberry Pi utilizado por el dispositivo. No desarrollé un kernel,

controladores ni un gestor de paquetes propios; aproveché esa base y personalicé los servicios, el arranque, la identidad y los modos de uso.

Esta decisión respondía al objetivo del proyecto. Escribir un sistema operativo completo no mejoraba de forma razonable la conversación ni las herramientas del agente. En cambio, el arranque en la interfaz adecuada y la recuperación ante fallos aportan valor directo.

Los servicios se inician y supervisan por separado. La pantalla puede cambiar de modo sin que por ello haya que borrar el contexto; el enlace con Android puede recuperarse sin reconstruir toda la interfaz. El nombre y los elementos visuales de ATLAS OS dan coherencia al dispositivo, pero no ocultan la base técnica de la que depende.

Preparé también una imagen distribuible del sistema. La imagen excluye secretos y datos de mi instalación personal, y su primer arranque permite que cada usuario configure sus propios accesos sin heredar los del desarrollador.



Figura 7. Escritorio de ATLAS OS en la pantalla del prototipo durante el desarrollo. Fotografía propia.

8.2. Modos de uso y recuperación local

El comando `atlas-screen` permite alternar entre la interfaz de ATLAS, el escritorio, la terminal y la recuperación. La vista nueva y la de depuración comparten la base conversacional. El modo oculto conserva la conversación y el audio bajo una imagen negra, mientras que el apagado de pantalla tiene otro efecto sobre el hardware.

Estas opciones no son solamente comodidades para desarrollar. Si la red o el proveedor fallan, el propietario necesita una forma de seguir trabajando con el equipo. La página local permite gestionar la conexión Wi-Fi y el teclado táctil facilita introducir datos sin depender de un teclado físico para cada operación.

Para los fallos más graves preparé RAFAS, Recovery Access For ATLAS Systems. Proporciona una ruta local de recuperación que no depende de que el modelo entienda una petición ni de que WebScreen funcione. Su propósito es devolver el control a una persona cuando las capas superiores no están disponibles.

En el prototipo, la recuperación física se configuró con facilidades administrativas pensadas para mis pruebas. Este modo permite acceder directamente al mantenimiento local del dispositivo.

8.3. Diseño digital de la carcasa

Diseñé la carcasa en FreeCAD con una forma triangular y bordes redondeados. El frontal y la trasera tienen una inclinación de 30° respecto a la vertical, con un vértice teórico de 60°. Quería un acabado negro sencillo, con la pantalla como elemento principal. Las primeras imágenes me ayudaron a definir el aspecto, pero para crear el modelo tuve que tener en cuenta las medidas de los componentes y el espacio de los conectores.

A principios de verano imaginé una primera propuesta visual de ATLAS Home V1. Esa imagen me sirvió como inspiración del producto, antes de elegir y montar los componentes. Su pantalla de 5 pulgadas y la indicación de IA local pertenecen a aquella idea inicial: el prototipo final utiliza una pantalla de 7 pulgadas y modelos externos.



Figura 8. Concepto inicial que imaginé al comenzar el trabajo y que me inspiró a principios de verano de 2026. Imagen conceptual, no modelo CAD ni montaje final.

Fui ajustando la distribución interna. Desplacé la Raspberry Pi hacia dentro para dejar espacio al HDMI, al USB del táctil y al cable del micrófono. Este último se conserva entero en el lateral derecho, separado de los altavoces frontales. También dejé ventilación, espacio para recoger el cable sobrante y huecos para llevar los puertos a la trasera mediante extensores.

En R04 reduje la carcasa a 190 × 163,2 × 144 mm, sin contar los pies ni el saliente del pulsador. Mantuve la pantalla real de 7 pulgadas y reduje el marco respecto a R03. Preparé vistas frontal, lateral y superior, secciones y un despiece para ver cómo encajaban las piezas. La distribución interna y los planos se pueden consultar en el anexo D, apartados D.2 y D.5.



Figura 9. Diseño de ATLAS A1 con la carcasa negra. Render del modelo 3D.

El montaje está dividido en cuatro piezas: cuerpo con base, frontal, tapa trasera y aro del micrófono. Se unen con tornillos y tuercas cautivas para poder abrirlo cuando sea necesario. La Pi utiliza tornillos M2,5 y las tapas M3. La pantalla mantiene sus cuatro anclajes. El encendido se resuelve en el diseño con un pulsador momentáneo conectado a J2 de la Pi.

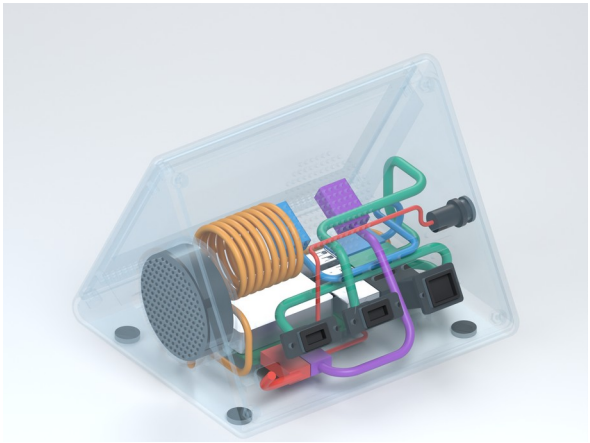


Figura 10. Distribución interna y reservas de cableado del CAD R04. Vista con la carcasa transparente. Elaboración propia.

8.4. Configuración para Bambu Lab H2S

La impresora del instituto es una Bambu Lab H2S, con un volumen de impresión de 340 × 320 × 340 mm. Aunque permite varios colores, preparé la carcasa completamente en negro. Utilicé Bambu Studio para dividir las piezas en capas y calcular los recorridos de impresión. [Bambu Lab, H2S]

En R05 preparé un perfil de PETG negro, boquilla de 0,4 mm y capas de 0,20 mm, con cuatro paredes, cinco capas superiores e inferiores y un relleno gyroid del 20 %.

El diseño quedó dividido en cuatro piezas y seis probetas para verificar los encajes en el modelo digital. Bambu Studio estimó 15 h 47 min y 367,1 g para la carcasa, sin contar las probetas. El laminado, las piezas y el montaje se detallan en el anexo D, apartados D.3 y D.4.

8.5. Presupuesto del prototipo

El coste directo aproximado del prototipo fue de 225 €. La cifra incluye los componentes principales que necesité para construir ATLAS A1. La impresión 3D no tuvo coste para mí porque utilicé la impresora y el filamento del instituto.

Elemento	Procedencia	Coste aproximado
Impresión 3D	Impresora y filamento del instituto	0 €
Micrófono USB	Comprado para el prototipo	15 €
Raspberry Pi 5 de 4 GB y Active Cooler	Comprados para el prototipo	125 €
Pantalla táctil SunFounder TS7 Pro	Comprada para el prototipo	85 €
Total		225 €

Los cables HDMI y USB, la tornillería y los adaptadores ya estaban incluidos con los componentes o se reutilizaron, por lo que no los contabilicé como compras separadas. El precio de la pantalla es orientativo y se redondeó a partir del precio consultado para la TS7 Pro.

8.6. Ampliaciones físicas y criterio de selección

Durante el diseño consideré sensores de presencia, iluminación o temperatura ambiental, e incluso un microcontrolador para tareas específicas. No los añadí solo porque la placa tuviese conexiones disponibles: cada sensor exige alimentación, cableado, software y una explicación de su utilidad.

La prioridad fue terminar una base coherente: escuchar, responder, mostrar estado y ejecutar acciones autorizadas. Las ampliaciones deben resolver un problema concreto que se pueda evaluar. Por ejemplo, un sensor podría ayudar a adaptar una función al entorno, pero eso necesitaría su propia prueba y no mejoraría automáticamente la comprensión del lenguaje.

III. EVALUACIÓN — Resultados y límites

9. Evaluación del prototipo

9.1. Medición de la experiencia completa

La pregunta de investigación no se podía responder únicamente comprobando que ATLAS generaba texto. Necesitaba valorar si escuchaba una petición, realizaba la acción adecuada, respondía y quedaba preparado para seguir utilizándose. Un turno que empieza rápido pero ejecuta una tarea equivocada no es un buen resultado.

Separé varios instantes de medida: final de la intervención, envío al modelo, primer fragmento recibido, inicio de reproducción y entrega del resultado útil. Según la prueba, el punto de partida fue una transcripción ya disponible o una entrada real por micrófono. Esos tiempos no se deben mezclar como si midiesen exactamente lo mismo.

La latencia inicial indica cuándo aparece la primera respuesta. La latencia del resultado indica cuándo llega el dato o la acción solicitada. Un preámbulo puede mejorar la primera sin reducir la segunda. También observé errores de transcripción, interrupciones, reconexiones y repetición de acciones, porque influyen en la fiabilidad de toda la secuencia.

Las pruebas automatizadas ayudan a repetir casos conocidos, pero tienen un alcance limitado. Pueden comprobar que se ignora un evento tardío o que se devuelve un error correcto; no certifican que una voz se haya oído bien en una habitación. Por eso distingo pruebas de contrato, integración, navegador y dispositivo físico.

9.2. Resultados de las mediciones históricas

Las primeras mediciones sirvieron sobre todo para localizar cuellos de botella. En una serie histórica con Realtime y delegación a otro agente, el primer audio de una consulta de almacenamiento tuvo una mediana de 0,304 segundos. Sin embargo, la respuesta con el resultado útil tuvo una mediana de 16,511 segundos. La reacción era rápida; la tarea seguía siendo lenta.

La mayor parte de esa espera correspondía a la delegación. El caso fue importante porque evitó una conclusión superficial: no podía decir que había resuelto la latencia solo porque el agente empezase a hablar en unas décimas. Había que seguir midiendo hasta la información que el usuario realmente había pedido.

Una consulta de hora mostró el mismo problema desde el lado contrario. Una función local podía resolverla de forma directa, mientras que una ruta que delegaba innecesariamente llegó a tardar más de diecisiete segundos en aquella prueba. Eso motivó el uso de herramientas más directas y, posteriormente, de rutinas para resultados previsibles.

Estas cifras pertenecen a etapas anteriores. La red, los modelos y las condiciones no fueron idénticos en todos los ensayos, por lo que no las utilizo como una comparación controlada de la versión actual. Los datos ampliados, tamaños de muestra y limitaciones se conservan en el anexo B, apartados B.2 a B.4, como evidencia del proceso de desarrollo.

9.3. Pruebas de voz, rutinas y Android

En una tanda de seis turnos con WebRTC se registraron tiempos entre 1,214 y 3,075 segundos desde la entrada textual reconocida hasta el inicio del audio remoto. La sesión utilizaba el modelo Realtime y contexto del proyecto, pero se inyectó texto para aislar parte del recorrido. Por tanto, esa prueba no mide cuánto tardó en reconocerse la palabra de activación ni demuestra escucha física por el altavoz.

En una comprobación de `atlas-chat`, una consulta con herramientas produjo el primer texto en 2,68 segundos y completó el turno en 3,01 segundos. Es una evidencia útil sobre ese recorrido de texto, no un tiempo de conversación hablada. Estas diferencias de método acompañan los resultados, en lugar de quedar escondidas en una nota genérica.

Las rutinas simples confirmaron que se podía evitar trabajo innecesario del modelo. En las pruebas registradas, el motor local resolvió una ejecución entre 8,4 y 10,9 milisegundos, mientras que la llamada HTTP completa tomó 53,9 milisegundos. A ese recorrido hay que añadir la síntesis y reproducción si el usuario espera una respuesta hablada.

En Android se comprobaron operaciones no intrusivas y un recorrido visual con capturas, lectura de elementos, gestos, escritura y parada. Una muestra registró el inicio del control en 192,2 milisegundos y una captura en 471,0 milisegundos. Estos números describen operaciones concretas, no la duración completa de cualquier tarea dentro del teléfono.

También se comprobó que un error recuperable de selección no cerrase automáticamente toda la sesión. Eso permitió corregir un paso sin volver a iniciar el control desde cero. La verificación de llamadas o mensajes reales quedó fuera de esas pruebas. El anexo B detalla esta separación entre funcionalidad disponible y evidencia obtenida.

9.4. Baterías de preguntas y mejora de las acciones

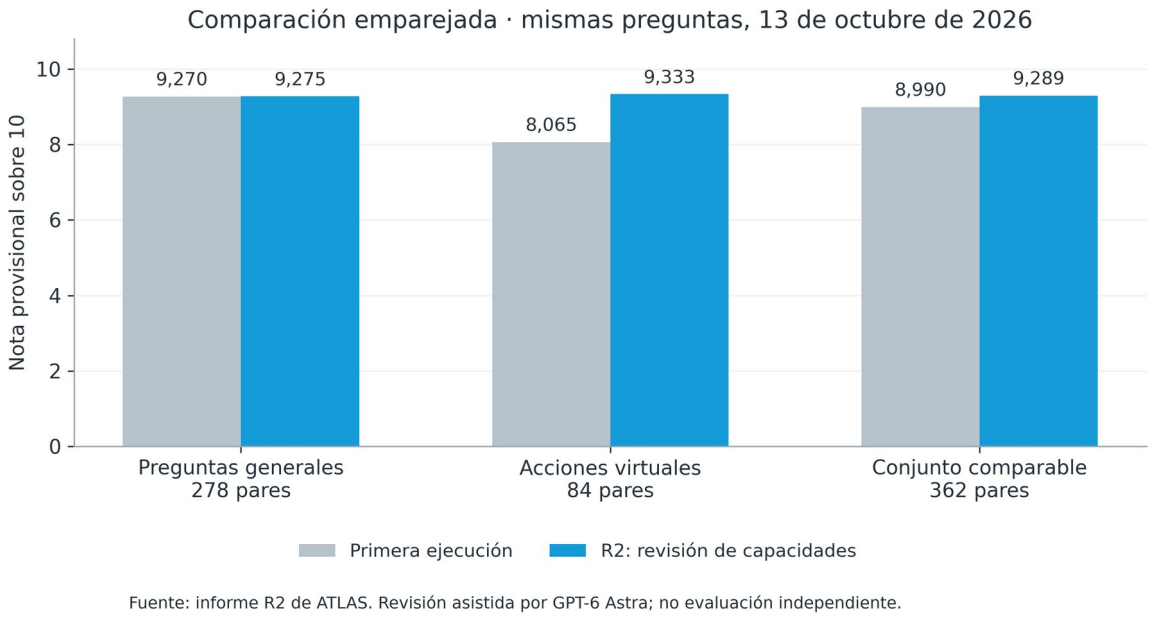
El 13 de octubre preparé 300 preguntas generales y 100 acciones, como cambiar el volumen o reproducir música. Introduje las preguntas por texto, con una salida de voz silenciosa y acciones virtuales. Medí el inicio del motor de voz, no la primera palabra audible.

Después de revisar los fallos, mejoré las instrucciones y los comandos y repetí las mismas 400 peticiones. En los 362 casos comparables, la nota pasó de 8,990 a 9,289 sobre 10. La mayor mejora estuvo en las acciones: de 8,065 a 9,333 en 84 pares. Las notas se revisaron con ayuda de GPT-6 Astra, sin una evaluación ciega, y no comprueban acciones físicas reales.

El 17 de octubre ejecuté otras 400 preguntas: 100 fáciles, 100 medias, 100 difíciles y 100 interacciones. Incluí 70 conversaciones de varios turnos para comprobar referencias como "¿y cuánto queda?" sin repetir el sujeto. No generé voz y las acciones eran virtuales.

Se completaron los 400 turnos sin errores del proveedor o de transporte. La mediana hasta el primer texto fue de 1,361 segundos y el percentil 95, de 3,456 segundos. Las comprobaciones automáticas dieron 9,60/10; esta puntuación indica cuántas condiciones programadas se cumplieron, no es una valoración humana de las respuestas.

ATLAS respondió sobre un estado sin consultarlo, aplicó un volumen distinto al anunciado y dejó una transmisión incompleta. Distinguí estos fallos de los errores del simulador. El método y las comparaciones están en el anexo C, apartados C.2 y C.5 a C.7; las preguntas y respuestas representativas, en el apartado C.8.



Fuente: informe R2 de ATLAS. Revisión asistida por GPT-6 Astra; no evaluación independiente.

Figura 11. Comparación emparejada antes/después de la revisión R2. Notas de revisión asistida; las acciones fueron virtuales. Elaboración propia a partir de los registros.

9.5. Optimización de la rapidez y la calidad

La mejora de calidad no aceleró todas las respuestas: en las preguntas generales, la mediana de inicio de síntesis pasó de 1,444 a 1,994 segundos. Preparé ensayos de 20 preguntas para separar llegada del texto, preparación de la frase y arranque de voz. Compactar las instrucciones no dio una mejora fiable y produjo más preámbulos, así que descarté el cambio.

También probé un ajuste del arranque de voz y conversaciones con hasta 240 turnos previos. En esos sondeos no observé una degradación importante por el historial. El ajuste de voz tampoco mejoraba los resultados de forma consistente y, al repetir las acciones, perdió algunos aciertos. Finalmente mantuve la versión que había mejorado la calidad. Los ensayos de latencia y conversaciones largas se pueden consultar en el anexo C, apartados C.3 y C.4.

9.6. Límites de la evaluación

Las primeras medidas eran muestras individuales o series pequeñas. Después añadí las baterías de 400 preguntas, pero siguen midiendo aspectos distintos: inicio de síntesis, primer texto o ejecución de acciones virtuales. Por eso no agrupé todos los resultados en una sola media. Tampoco es lo mismo una nota revisada con ayuda de un modelo que una puntuación calculada mediante comprobaciones automáticas.

También hubo diferencias entre el entorno de pruebas y el uso diario. Una sesión recién preparada, un teléfono con otra aplicación en primer plano o una red distinta pueden modificar el resultado. El reconocimiento de voz depende además del micrófono, la distancia, el ruido y el comportamiento del navegador.

Estas campañas miden la respuesta en los entornos descritos; no sustituyen una medición del recorrido acústico completo, desde el final de la voz del usuario hasta la primera palabra audible. El anexo C recoge las pruebas realizadas, incluidos sus fallos.

El número de pruebas de software superadas tampoco debe utilizarse como sinónimo de calidad total. Demuestra cobertura de casos definidos, pero puede dejar fuera un error que nadie había previsto. Una conclusión defendible combina esas comprobaciones con observación real y explica el alcance de los resultados.

IV. CONCLUSIONES Y CONTINUIDAD DEL PROYECTO

10. Respuesta a la pregunta inicial

El prototipo permitió comprobar que era viable unir conversación, contexto, herramientas y un dispositivo físico pequeño. ATLAS dejó de ser una idea de chat y pasó a disponer de varios canales de interacción y de operaciones sobre su entorno. En ese sentido, la hipótesis principal quedó respaldada por el desarrollo realizado.

La confirmación tiene límites. El sistema depende de servicios externos para su capacidad conversacional principal, necesita supervisión en tareas complejas y no todas las funciones han recibido el mismo grado de validación. Por eso no concluyo que haya construido un agente infalible ni un producto comercial.

El cambio más importante fue sustituir la cadena inicial por una estructura centrada en Realtime y herramientas directas. OpenClaw aportó una base útil durante una etapa extensa, pero el flujo que había construido alrededor acumulaba demasiada espera para la experiencia que buscaba. Desarrollar una arquitectura propia permitió controlar mejor el recorrido sin renunciar a lo aprendido en el prototipo anterior.

11. Aprendizajes de la integración del sistema

La principal dificultad no fue únicamente escoger un buen modelo. Un agente útil necesita saber qué herramientas tiene, recibir resultados fiables y gestionar correctamente lo que ocurre entre una petición y la siguiente. Un fallo de audio o una acción incompleta pueden hacer que parezca poco inteligente aunque la interpretación inicial fuese correcta.

También aprendí que rapidez y autonomía deben evaluarse con cuidado. Hablar antes no significa terminar antes, y utilizar más herramientas no significa razonar mejor. Algunas tareas se benefician de un modelo flexible; otras se resuelven mejor con una rutina pequeña y predecible. Elegir entre ambas fue una parte central del diseño.

El hardware convirtió esas decisiones en algo tangible. La pantalla, el micrófono, los altavoces y el sistema operativo hicieron posible utilizar ATLAS como un dispositivo dedicado. Al mismo tiempo, las dependencias entre alimentación, audio y sesión

gráfica mostraron que no se podía tratar la electrónica como una simple lista de piezas conectadas.

La documentación también terminó formando parte del funcionamiento del agente. Organizar el conocimiento en archivos con responsabilidades claras ayudó a mantener su contexto y a explicar el proyecto. No obstante, una instrucción bien redactada no reemplaza una comprobación ni un permiso aplicado por software.

12. Desarrollo futuro

La carcasa quedó completada como un modelo tridimensional con cuatro piezas, ventilación, acceso a los puertos, distribución acústica y reservas para los cables. El diseño integra las cotas de la pantalla y el micrófono, el pulsador, los extensores y el cableado. Se documentó mediante renders, planos y medidas del modelo.

Si continuase con ATLAS, me gustaría ampliar sus puertos, interfaces y sensores, pero también pulir lo que ya tengo. Una prioridad sería utilizar un micrófono más pequeño y de mayor calidad, junto con altavoces independientes de la pantalla. El audio fue una de las partes que más problemas me dio: el micrófono podía volver a captar las respuestas del agente. Ese retorno es eco acústico, y los relojes independientes de captura y reproducción pueden dificultar su cancelación. Revisaría la posición de los componentes, su sincronización y la cancelación de eco como un conjunto (IETF, RFC 7874).

También diseñaría una carcasa más elegante y avanzada, con varios colores y materiales de mayor calidad, como el aluminio. Incorporaría una refrigeración más eficaz, sin descuidar el ruido del ventilador, el peso ni la recepción inalámbrica. Tendría que adaptar el montaje al nuevo hardware y comprobar de nuevo su comportamiento térmico y acústico.

Otra posibilidad sería incorporar una batería con un sistema como PiPower 5, para utilizar ATLAS sin estar siempre conectado a un enchufe. Antes tendría que medir el consumo del conjunto y comprobar la corriente disponible en modo batería. Su documentación recomienda apagar la Pi 5 al perder la alimentación externa si consume más de 3 A, por lo que no puedo dar por asegurada la autonomía del A1 completo. Con una alimentación adecuada y un apagado ordenado, también reduciría el riesgo de corrupción de archivos durante un corte de corriente (SunFounder, PiPower 5).

En las interfaces, seguiría puliendo el WebScreen y la experiencia de uso de ATLAS. También me gustaría mejorar y optimizar la aplicación de control para Android, y ampliar las capacidades de Android Use. Revisaría la selección de elementos, la navegación y la comprobación de los resultados, manteniendo los permisos y las confirmaciones en las acciones sensibles.

En la voz, buscaría una entonación y una dicción más naturales, con cambios de ritmo, pausas y énfasis adecuados a cada respuesta. Querría que ATLAS sonase más cercano, sin exagerar las emociones.

Por último, me gustaría integrar más modelos y proveedores de inteligencia artificial. La arquitectura está muy vinculada a gpt-realtime-2.1 de OpenAI, por lo que no bastaría con sustituir el nombre del modelo. Separaría mejor la conexión con cada proveedor y comprobaría los turnos de conversación, las herramientas, las imágenes y la voz en cada caso.

En software quedan ensayos prolongados de estabilidad, la medición del recorrido acústico completo con un protocolo de latencia homogéneo y una revisión de las funciones sensibles de Android bajo condiciones seguras. También será necesario completar el aislamiento efectivo de perfiles, la gestión de actualizaciones y una política clara de conservación y borrado de datos.

En el acceso remoto, me gustaría diseñar una aplicación de autenticación basada en TOTP, con códigos temporales para entrar en atlas.samilososami.com/try y atlas.samilososami.com/chat, sin depender únicamente de una contraseña estática (IETF, RFC 6238). Tendría que proteger el secreto que genera los códigos, limitar los intentos y preparar una recuperación segura si perdiese el teléfono. Antes de ofrecer el dispositivo a otras personas, también revisaría los permisos, la recuperación física y la configuración inicial.

El resultado es un sistema funcional y una investigación sobre cómo construirlo. Lo más valioso es mostrar las funciones integradas, la estructura que consigue una conversación más útil, las limitaciones identificadas y la forma de comprobar mejoras posteriores.

Referencias documentales

Las fuentes siguientes respaldan conceptos y especificaciones. Los resultados del prototipo proceden de sus registros y de la memoria de desarrollo, recopilados en los anexos B y C. La documentación inicial se consultó el 1 de octubre de 2026; las referencias añadidas después, el 18 y el 19 de octubre. Los enlaces permiten consultar las fuentes utilizadas.

ANDROID DEVELOPERS. *AccessibilityService*. Referencia oficial (https://developer.android.com/reference/android/accessibilityservice/AccessibilityService). Consulta: 1 de octubre de 2026.

ANDROID DEVELOPERS. *Foreground service types*. Documentación oficial (https://developer.android.com/develop/background-work/services/fgs/service-types). Referencia técnica del desarrollo.

DEBIAN PROJECT. *Debian 13 "trixie"*. Información de la distribución (https://www.debian.org/releases/trixie/index). Referencia de la base del sistema.

ELEVENLABS. *Latency optimization*. Guía técnica (https://elevenlabs.io/docs/eleven-api/guides/how-to/best-practices/latency-optimization). Referencia de síntesis y transmisión de audio.

GONZÁLEZ KAMEL, Sami. *ATLAS: memoria inicial y registros de desarrollo*. Material propio: documento original, historial del proyecto y pruebas recogidas en los anexos. 2026.

GONZÁLEZ KAMEL, Sami. *ATLAS*. Repositorio del proyecto (https://github.com/samilososami/ATLAS). Código, manuales y documentación de desarrollo; contiene componentes de terceros identificados por sus respectivas licencias.

MANUAL DEL CENTRO. *Dosier del Treball de Recerca*. Fotografías facilitadas el 1 de octubre de 2026. Páginas aportadas: índice, 2–3, 12, 14–25, 27–30 y 34–35. Traducción de consulta: .atlas/tdrman/MANUAL-TDR-ES.md.

NETWORKMANAGER PROJECT. *Manual pages*. Manuales de red (https://networkmanager.dev/docs/man-pages/). Referencia técnica del desarrollo.

OPENAI. *GPT Realtime 2.1*. Ficha del modelo (https://developers.openai.com/api/docs/models/gpt-realtime-2.1). Consulta: 1 de octubre de 2026.

OPENAI. *Realtime conversations*. Guía de conversaciones (https://developers.openai.com/api/docs/guides/realtime-conversations). Referencia de sesiones, audio y herramientas.

OPENAI. *Prompting Realtime models*. Guía de instrucciones (https://developers.openai.com/api/docs/guides/voice-prompting). Referencia del diseño conversacional.

OPENAI. *Understanding and counting tokens*. Explicación del recuento de tokens (https://help.openai.com/es-es/articles/4936856-understanding-and-counting-tokens). Consulta: 1 de octubre de 2026.

OPENCLAW. *OpenClaw Docs*. Documentación general (https://docs.openclaw.ai/). Referencia de la etapa inicial, no dependencia de la arquitectura actual.

RASPBERRY PI LTD. *Raspberry Pi 5*. Características oficiales (https://www.raspberrypi.com/products/raspberry-pi-5/). Consulta: 1 de octubre de 2026.

RASPBERRY PI LTD. *Configuration*. Documentación del sistema (https://www.raspberrypi.com/documentation/computers/configuration.html). Referencia técnica del montaje.

SUNFOUNDER. *TS7-Pro: Hardware introduction*. Documentación de la pantalla (https://docs.sunfounder.com/projects/ts7-pro/en/latest/hardware_introduction.html). Consulta: 1 de octubre de 2026.

TAILSCALE. *Connection types*. Conexiones directas y retransmitidas (https://tailscale.com/docs/reference/connection-types). Consulta: 1 de octubre de 2026.

WEB AUDIO COMMUNITY GROUP. *Web Speech API*. Especificación (https://webaudio.github.io/web-speech-api/). Referencia del reconocimiento y síntesis en navegador.

WHISKEYSOCKETS. Baileys. Cliente de WhatsApp y documentación del proyecto. https://github.com/WhiskeySockets/Baileys Consulta: 18 de octubre de 2026.

BAMBU LAB. H2S: características y volumen de impresión. https://blog.bambulab.com/h2s-the-ultimate-single-nozzle-3d-printer-now-bigger-than-ever/ Consulta: 18 de octubre de 2026.

BAMBU LAB. Bambu Studio. Código y documentación del laminador. https://github.com/bambulab/BambuStudio Consulta: 18 de octubre de 2026.

FREECAD. Proyecto y documentación del modelador paramétrico. https://www.freecad.org/ Consulta: 18 de octubre de 2026.

RASPBERRY PI LTD. Active Cooler. Refrigeración de la placa. https://www.raspberrypi.com/products/active-cooler/ Consulta: 18 de octubre de 2026.

GONZÁLEZ KAMEL, Sami. ATLAS: batería de 400 interacciones, 17 de octubre de 2026. https://github.com/samilososami/ATLAS/tree/main/benchmarks/20261017-interactions Consulta: 18 de octubre de 2026.

GONZÁLEZ KAMEL, Sami. ATLAS: catálogo de expresiones y renderizador. https://github.com/samilososami/ATLAS/tree/main/expressions Consulta: 18 de octubre de 2026.

GONZÁLEZ KAMEL, Sami. ATLAS: código fuente y documentación pública del proyecto. https://github.com/samilososami/ATLAS Consulta: 18 de octubre de 2026.

GONZÁLEZ KAMEL, Sami. ATLAS: web de presentación y portal protegido. https://atlas.samilososami.com/ Consulta: 18 de octubre de 2026.

AMAZON (2020). Introducing the All-New Echo Family—Reimagined, Inside and Out. Consulta: 19 de octubre de 2026.

GOOGLE (2019a). Nest Mini brings twice the bass and an upgraded Assistant. Consulta: 19 de octubre de 2026.

GOOGLE (2019b). Get help at a glance with Google Nest Hub. Consulta: 19 de octubre de 2026.

APPLE (2020). Apple introduces HomePod mini: A powerful smart speaker with amazing sound. Consulta: 19 de octubre de 2026.

IETF. RFC 7874. WebRTC Audio Codec and Processing Requirements, sección 5: cancelación de eco acústico. Consulta: 19 de octubre de 2026.

SUNFOUNDER. PiPower 5: características de la placa. Consulta: 19 de octubre de 2026.

SUNFOUNDER. PiPower 5: software, alimentación y apagado seguro. Consulta: 19 de octubre de 2026.

IETF. RFC 6238. TOTP: Time-Based One-Time Password Algorithm. Consulta: 19 de octubre de 2026.